

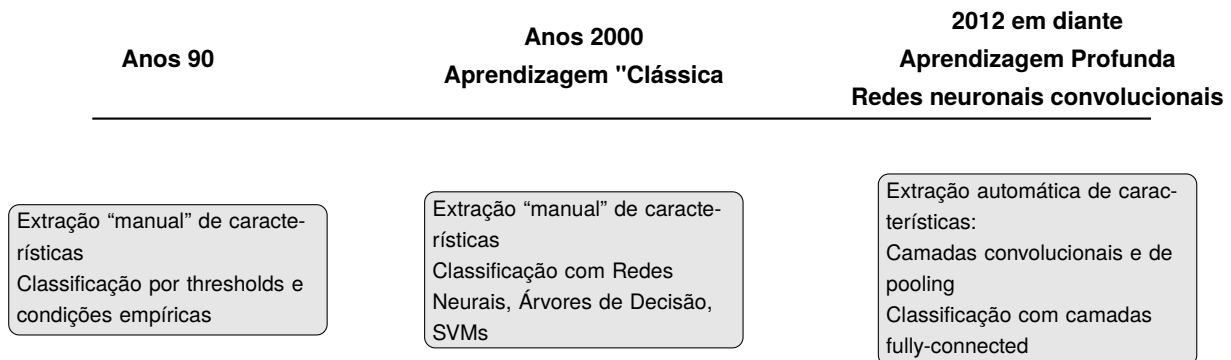
Aprendizagem Profunda para Visão por Computador

O QUE É VISÃO POR COMPUTADOR?

Área multidisciplinar onde o objetivo é extrair informações de imagens e vídeos de forma a compreender o conteúdo para realizar tarefas úteis. Estas podem ser:

1. Classificação (binária ou multiclasse) - resultado é uma classe
2. Regressão - resultado é a previsão de um valor contínuo

Os computadores vêm números, onde numa imagem são tipicamente três matrizes [shape $\rightarrow$ (x, y, 3)]. As características retiradas da imagem (features) variam consoante o domínio do problema e não são propriamente fáceis de obter através de cálculos. (por exemplo: obter o nariz e boca de uma pessoa; faróis, rodas de um carro; portas, janelas de uma casa)



ESPAÇOS DE CORES E OPENCV

O **OpenCV** é uma *library* que inclui algoritmos de processamento e análise de imagens, onde estas são carregadas e processadas como matrizes. As funções estão otimizadas para operar sobre matrizes e deve-se, por isso, evitar o processamento pixel-pixel. **OpenCV usa representação BGR para imagens a cores.**

Algumas funções desta biblioteca:

- `img = cv2.imread("images/flower.jpg")` $\rightarrow$ ler a imagem (BGR)
- `imgGray = cv2.imread("images/flower.jpg", cv2.IMREAD_GRAYSCALE)` $\rightarrow$ ler a imagem em tons de cinzento
- `cv2.imwrite("images/flower.png",img)` $\rightarrow$ escrever imagem no disco
- `cv2.imshow("Imagem de teste", img)` $\rightarrow$ mostrar a imagem

A **representação das imagens** pode ser policromática (imagens com 3 canais de cores) ou monocromática (imagens com um único canal de luminância, em níveis de cinzento)

Alguns dos espaços de cores mais conhecidos são os seguintes:

1. **RGB/BGR** $\rightarrow$ espaço de cores mais usado (em computadores, fotografias digitais, etc.), onde a cor de cada pixel é composta por 3 canais com valores entre 0 e 255 - R, G, B
2. **YCbCr** $\rightarrow$ espaço de cores usado em transmissão de vídeo/TV, em que há 3 valores por pixel: um de luminância/tons de cinzento (**Y**) e os outros 2 canais com crominâncias de azul e vermelho (**Cb** e **Cr**), respetivamente
3. **HSV** $\rightarrow$ espaços de cores popular em aplicações de visão por computador. Há 3 canais entre os quais: Hue que indica a cor (vermelho, amarelo, ..., rosa) numa escala de 0 a 360 graus; Saturation que nos diz se a cor é mais viva ou esbatida; Value diz se é um tom mais claro ou escuro (no **HSL**, semelhante ao HSV mas com uma componente de Lightness em vez do Value - ver Figura 2)
No **OpenCV** a componente Hue assume valores de 0 e 180.

Os valores de cada componente da cor podem assumir diferentes gamas de representação: em inteiros de 8 bits sem sinal $\rightarrow$ [0, 255]; inteiros de 8 bits com sinal $\rightarrow$ [-128, 127] (no caso dos canais **Cb** e **Cr**); valores reais entre -1 e 1 ou 0 e 1.

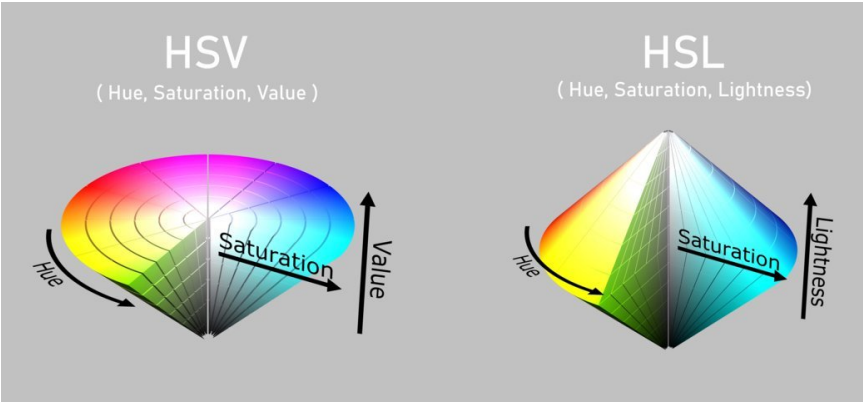


Figura 1: Comparação entre os modelos de cores HSL (Hue, Saturation, Lightness) e HSV (Hue, Saturation, Value).

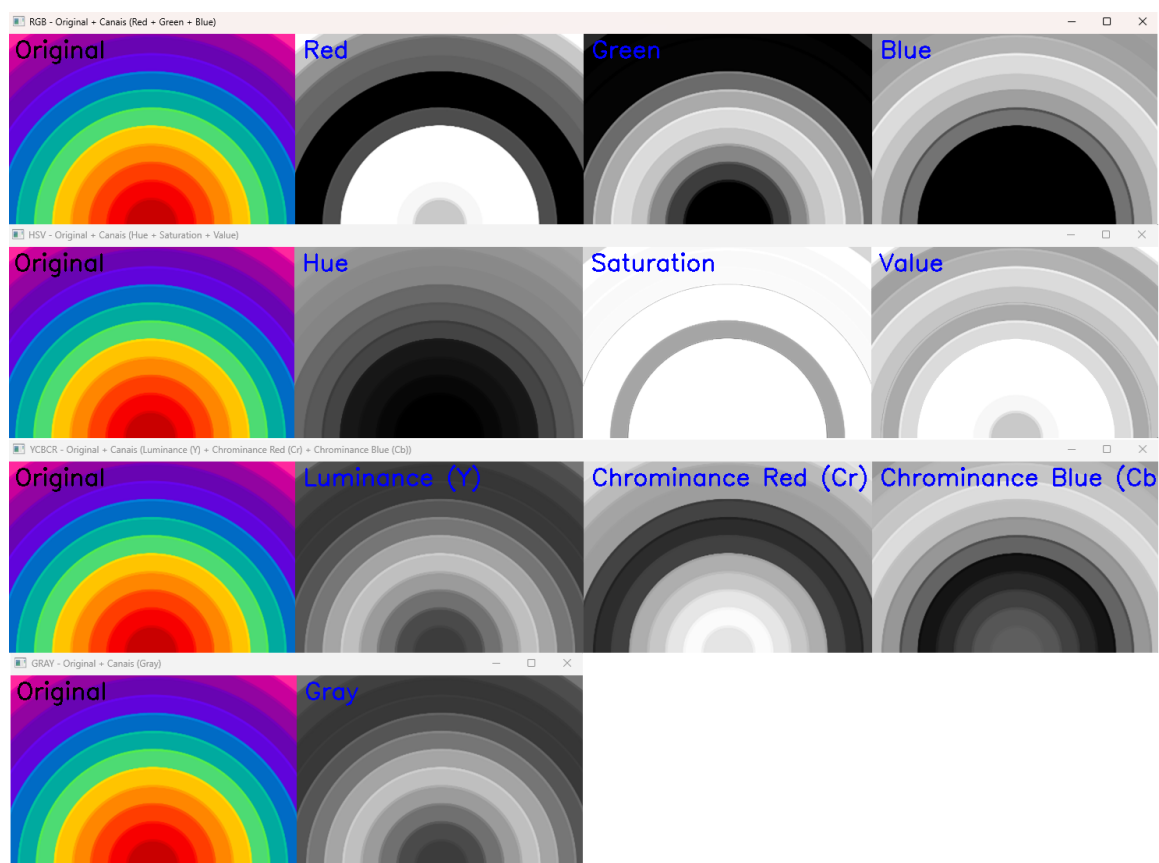


Figura 2: Exemplo de imagens com vários espaços de cores diferentes

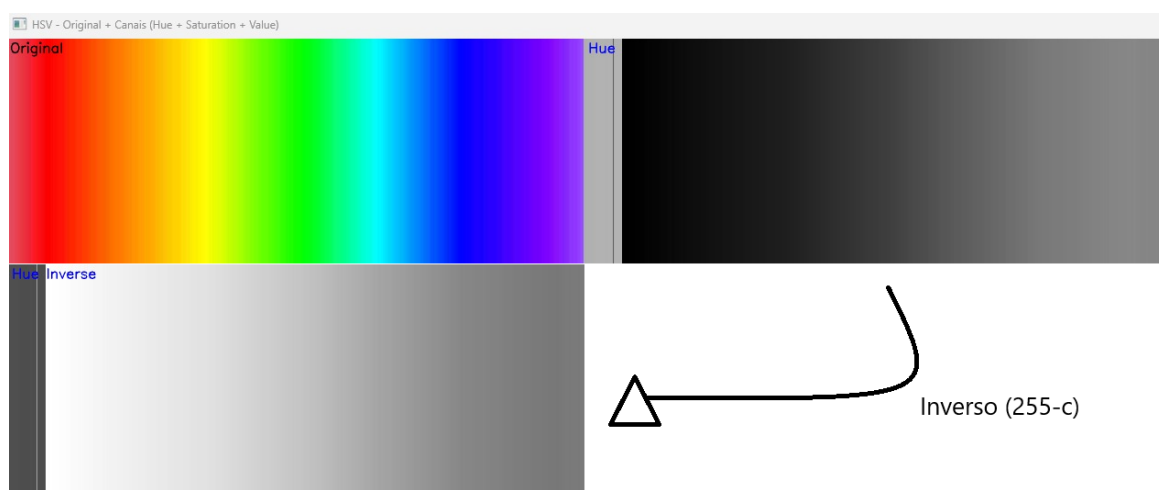


Figura 3: Espectro de cores com canal Hue do HSV e o seu inverso

A **binarização** consiste em atribuir valores binários aos pixels da imagem, ou seja, representá-la em duas cores - branco ou preto (0/1 ou 0/255). Serve para localizar objetos específicos, separar elementos de background e foreground e preparar imagens de texto para OCR.

Para realizarmos binarização normalmente utilizamos um valor de limiar/**threshold**, em que será 0 ou 1 consoante o valor do pixel seja maior ou menor que o valor do threshold.

No **OpenCV** pode ser feita através de:

- $(t, \text{mask2}) = \text{cv2.threshold}(\text{img}, 100, 255, \text{cv2.THRESH_BINARY}) \rightarrow$ gera uma máscara por comparação com um valor de limiar (no exemplo, acima de 100 transforma em branco/255) – tipicamente usada para processar um único canal (tons de cinzento); tem várias opções quanto ao output
O terceiro argumento numérico da função `threshold(...)` é o valor a atribuir à máscara quando a operação dá valor lógico 1
- $\text{mask1} = \text{cv2.inRange}(\text{img}, 100, 200) \rightarrow$ gera uma máscara com base em dois limiares (min e max); aos valores de pixel entre min e max corresponderá o valor lógico 1, aos restantes 0

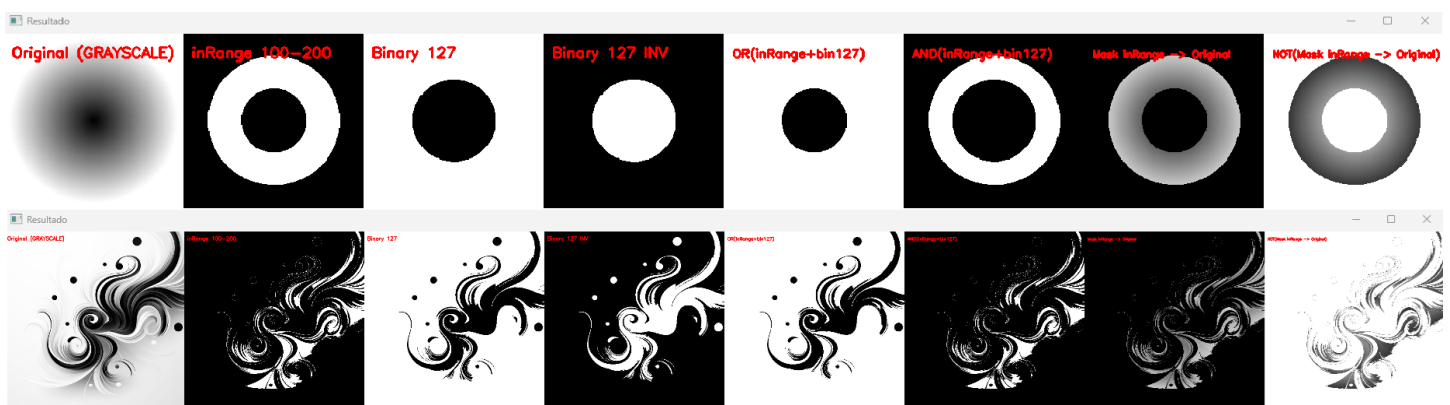


Figura 4: Exemplo de imagens com máscaras

Na Figura 4 conseguimos observar várias máscaras a partir de uma imagem importada com GRAYSCALE¹. Seguida dessa imagem temos mais 7 imagens "transformadas", sendo estas:

1. **inRange (100-200)** $\rightarrow \text{cv2.inRange}(\text{img}, 100, 200)$, que não capta tanto os pretos e os brancos e sim mais os tons de cinzento da imagem
2. **Binary 127** $\rightarrow \text{cv2.threshold}(\text{img}, 127, 255, \text{cv2.THRESH_BINARY})$, que através de um threshold de 127 define os brancos e pretos da imagem
3. **Binary 127 INV** $\rightarrow \text{cv2.threshold}(\text{img}, 127, 255, \text{cv2.THRESH_BINARY_INV})$, inverso de Binary 127, ou seja, altera o sentido do sinal da condição
4. **OR(inRange+bin127)** $\rightarrow \text{cv2.bitwise_or}(\text{mask_range}, \text{mask_bin})$, faz a união da máscara do inRange e Binary 127 (como se o threshold passasse a ser a partir de [100, 255])
5. **AND(inRange+bin127)** $\rightarrow \text{cv2.bitwise_and}(\text{mask_range}, \text{mask_bin})$, faz a interseção da máscara do inRange e Binary 127
6. **Mask inRange $\rightarrow$ Original** $\rightarrow \text{masked_img} = \text{cv2.bitwise_and}(\text{img}, \text{img}, \text{mask}=\text{mask_range})$, combina a máscara do inRange à imagem Original
7. **NOT(Mask inRange $\rightarrow$ Original)** $\rightarrow \text{mask_img_not} = \text{cv2.bitwise_not}(\text{masked_img})$, é o complemento/negativo da máscara aplicada do Mask inRange $\rightarrow$ Original

¹através da função $(\text{img} = \text{cv2.imread}(\text{path}, \text{cv2.IMREAD_GRAYSCALE}))$

São **operações não lineares** que podem ser aplicadas a imagens binarizadas (em preto/branco) ou imagens a tons de cinzento. O principal objetivo da aplicação a imagens/máscaras binárias é:

- remover potencial ruído
- preencher buracos em componentes conexos de pixels brancos (255)

Um **elemento estruturante** é um "template", que tem dimensões bastante menores que a imagem original, e que se desloca pela imagem para aplicar as operações morfológicas. Este elemento estruturante pode ter várias formas geométricas, como elipses (`cv2.MORPH_ELLIPSE`), retângulos (`cv2.MORPH_RECT`), cruzeiros (`cv2.MORPH_CROSS`), etc.

As **operações morfológicas** existentes são divididas em dois conjuntos:

- **Operações elementares** (pode ser visto na Figura 5):
 - **Erosão** → "**Encolhe**" objetos, **remove** objetos pequenos e **pode separar** objetos.
 - **Dilatação** → "**Alarga**" objetos, **preenche** buracos no interior dos objetos e **pode ligar** objetos próximos.

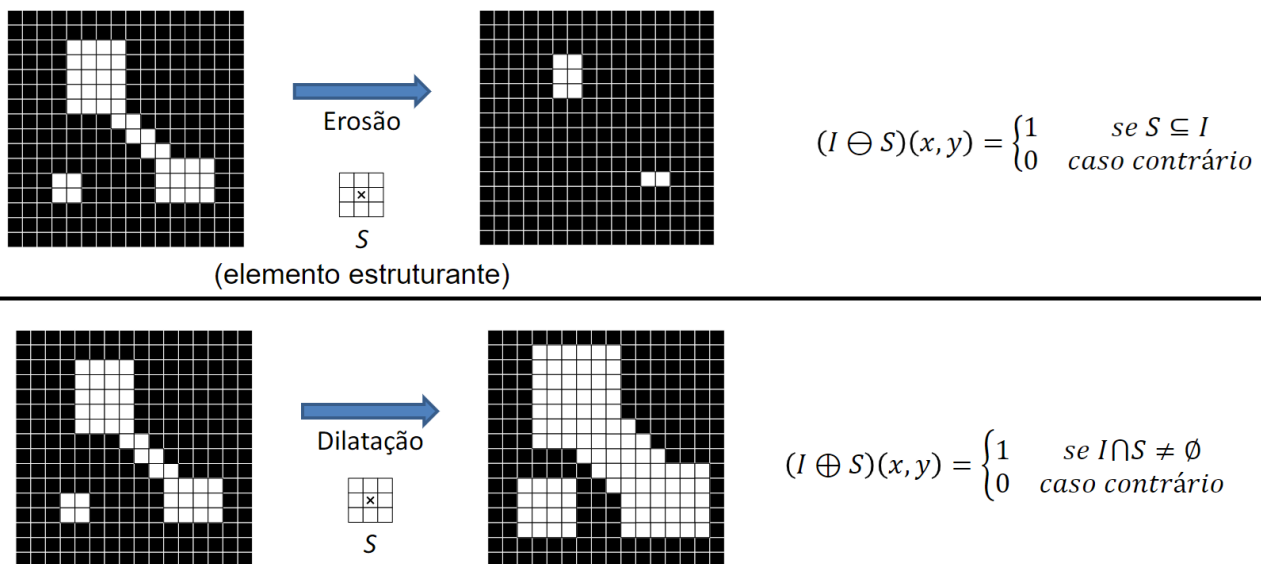


Figura 5: Erosão e Dilatação: explicação visual

- **Operações compostas:**
 - **Abertura** → É uma erosão seguida de uma dilatação.

$$I \circ S = (I \ominus S) \oplus S$$

Remove ruído **fora** dos objetos e **remove** objetos pequenos; pode **separar** objetos unidos por ligações finas, por conta da erosão (1ª operação).

- **Fecho** → É uma dilatação seguida de uma erosão.

$$I \bullet S = (I \oplus S) \ominus S$$

Remove ruído **dentro** dos objetos e **preenche buracos** dentro dos objetos; pode **juntar** objetos próximos, por conta da dilatação (1ª operação).

Para perceber melhor estas operações, a Figura 6 demonstra as operações morfológicas mencionadas, utilizando o OpenCV. As figuras representam a imagem original, a binarização da imagem original, uma erosão, uma dilatação, uma abertura, um fecho, um gradiente morfológico/fronteiras e uma abertura seguida de um fecho, respetivamente, da esquerda para a direita. Para a mesma imagem foram aplicadas duas kernels para detetar diferenças de "preenchimentos".

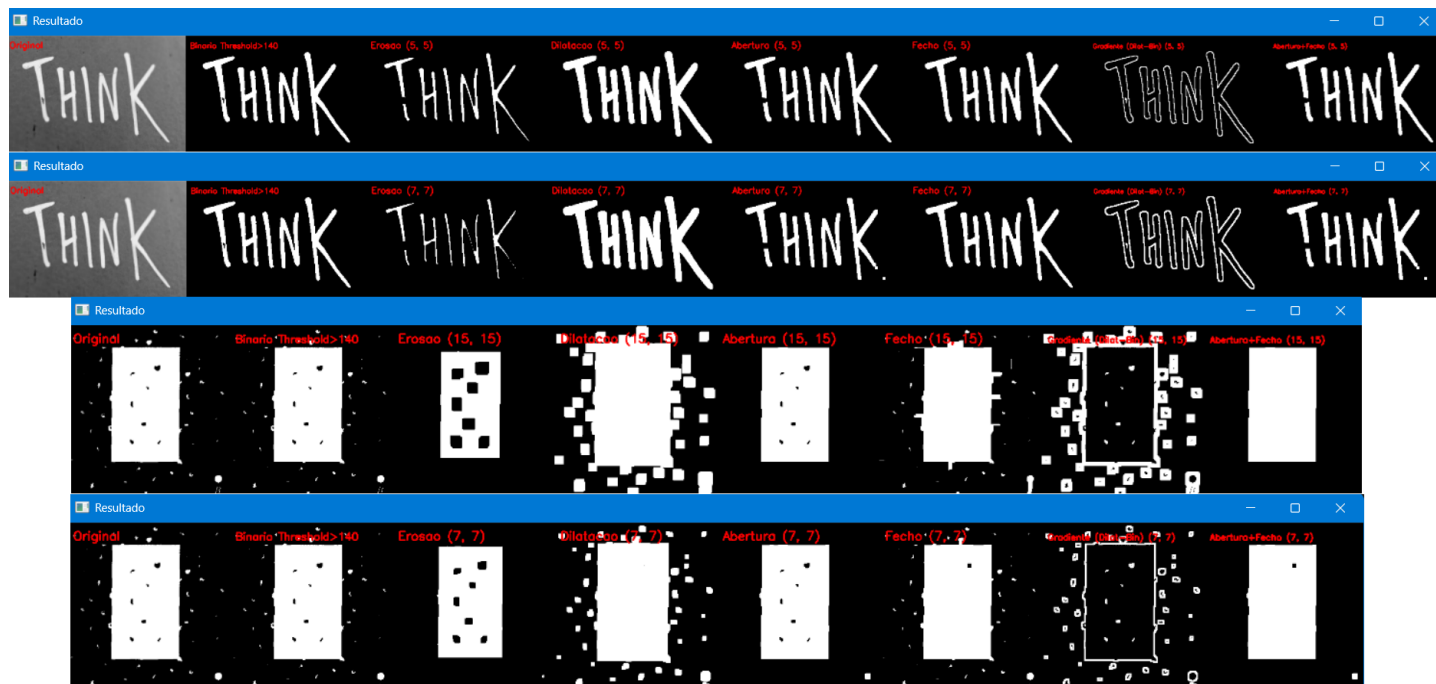


Figura 6: Exemplo de imagens com morfologias e tamanhos de elementos estruturantes diferentes

EXEMPLO COM MÁSCARAS E MORFOLOGIA

Uma aplicação prática das máscaras pode ser observada na Figura 7. Ao importar uma imagem em tons de cinza (GRAYSCALE) e aplicar uma máscara baseada em um limiar (threshold) específico, é possível refinar a segmentação com técnicas de morfologia — como a aplicação de dois fechos, representação ilustrada na figura. Com a máscara obtida, podemos então aplicá-la à imagem original para identificar e contar as moedas presentes, bem como "estimar" o valor monetário total representado.

Além disso, podem ser utilizadas máscaras mais complexas, como aquelas baseadas no espaço de cores HSV, para segmentar diferentes tonalidades. Isso permitiria, por exemplo, distinguir moedas de diferentes valores e identificar quantas existem de cada tipo, tornando a abordagem altamente útil em aplicações do mundo real, como contagem de dinheiro automática e classificação de moedas.

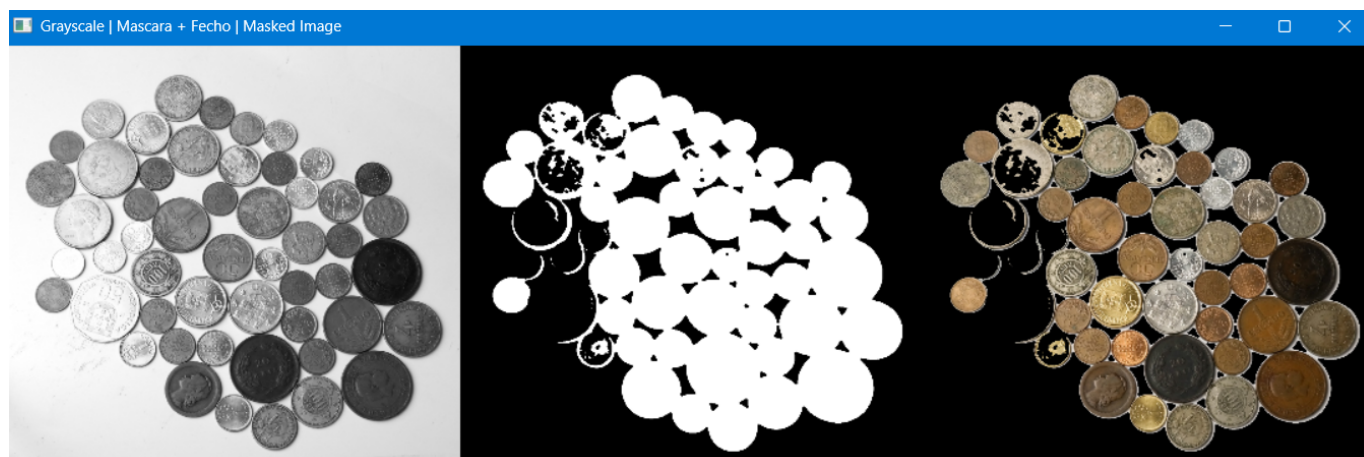


Figura 7: Exemplo prático com máscaras e morfologia

NOTA: Este caso tem algum ruído em algumas moedas na extremidade esquerda da figura, que confunde com o fundo e que não foi tratado para a realização deste exemplo.

CONVOLUÇÕES

É uma **operação linear** bastante importante para processamento de sinal. Na Figura 8 podemos ver os cálculos e a matriz originada ao aplicar uma convolução com um kernel específico.

A convolução serve para tarefas como , estimação do gradiente, aumentar a nitidez/contraste de uma imagem, etc.

- **Estimação do gradiente** → Convolução com kernel de Sobel através de:

– **Direção horizontal** (componente x do gradiente G_x):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

– **Direção vertical** (componente y do gradiente G_y):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Gradiente de imagem (∇I): Derivadas ao longo das direções horizontal e vertical.

Magnitude do Gradiente → $G_{max} = \sqrt{G_x^2 + G_y^2}$, que depois pode-se aplicar Binarização de Otsu e Algoritmo de Canny.

A binarização de Otsu é um método adaptativo onde o valor de limiar (threshold) para binarização é obtido automaticamente. Este valor de limiar corresponde ao valor que separa as classes de primeiro plano (foreground) e fundo (background), minimizando a variância intra-classe dos valores de luminância dos pixels. A Figura 9 mostra a aplicação da máscara de Otsu ao exemplo prático das moedas, da Figura 7. Podemos observar que ele consegue detectar as moedas à esquerda — que não eram captadas no exemplo anterior —, mas também inclui partes do fundo, devido à pouca separação e distinção nos tons de luminância entre as tais moedas no plano esquerdo e o background.

- **Filtragem de ruído** → exemplo: Lenna com ruído e aplicando uma convolução com kernel gaussiano 3x3

- **Aumentar a nitidez de imagens** → Sharpen com uma kernel personalizada (exemplo: A persistência da memória)

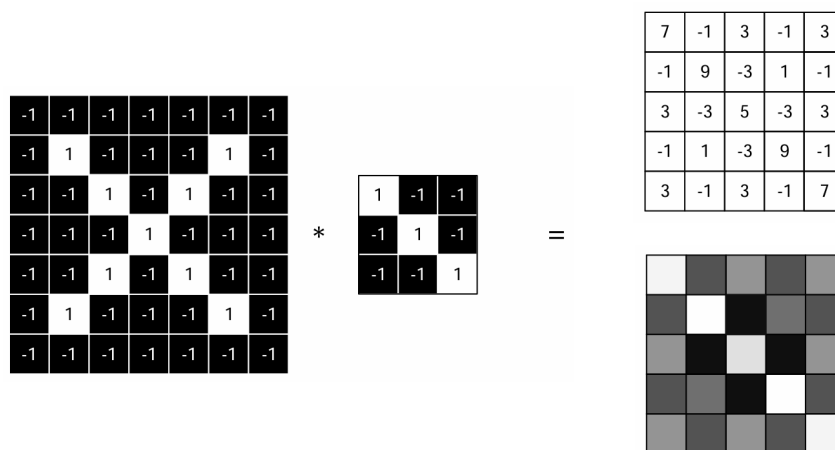


Figura 8: Convolução passo-a-passo

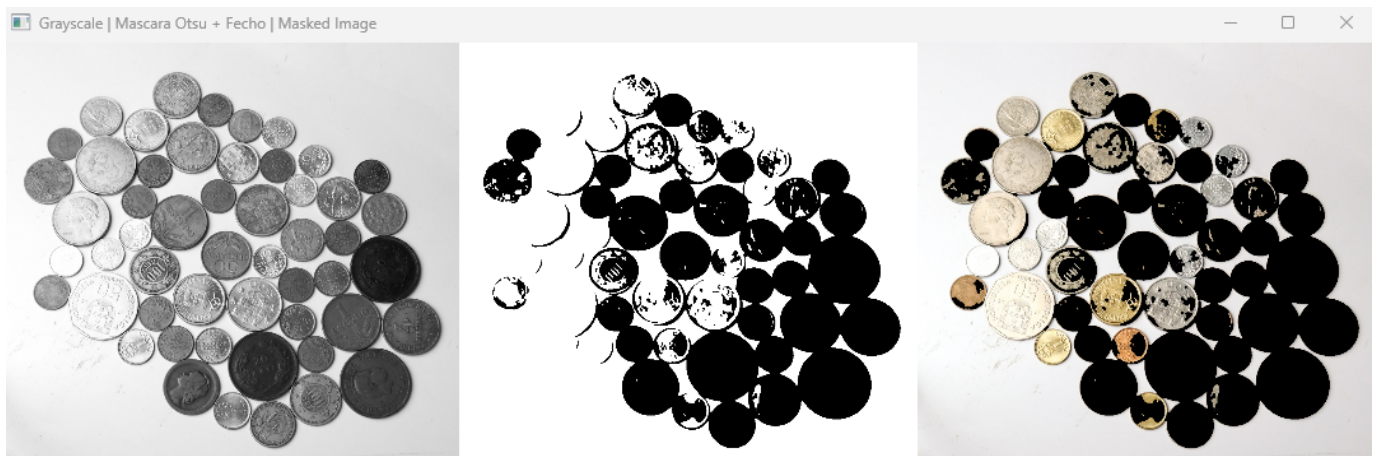


Figura 9: Exemplo da Máscara de Otsu com um Fecho

No OpenCV é possível obter estatísticas sobre as regiões em branco/componentes conexas da imagem através da função:

(numLabels, labels, boxes, centroids) = cv2.connectedComponentsWithStats(imgBin)

Cada uma das variáveis representa o seguinte:

- numLabels – número de componentes conexos
- labels – Id's dos componentes
- boxes – *bounding boxes* - coordenadas do canto superior esquerdo, altura e largura dos retângulos envolventes
- centroids – coordenadas dos centróides dos componentes conexos

REDES NEURONAIS CLÁSSICAS

As **redes neurais** são inspiradas na biologia e são estruturas compostas por neurónios e organizadas por camadas. Estas redes aceitam valores dos atributos (*features*) para produzirem predições de classes ou estimativas de valores, em problemas de classificação e regressão, respetivamente.

Cada **neurónio** realiza uma "soma pesada" que é o produto interno entre os valores de entrada X_i e os pesos associados às ligações w_i . No fundo isto é:

$$y = g(w_0 + \sum_{i=1}^n w_i \cdot x_i)$$

em que w_0 é um valor de *bias* e $g(z)$ é uma função de ativação.

A **função de ativação** é crucial porque, sem a presença desta, a separação dos dados será sempre linear, ou seja, separa os dados e as classes por uma linha reta, do tipo $y = m \cdot x + b$. Já com funções de ativação, conseguem-se separações não lineares e que, por conseguinte, ajustam-se melhor aos dados que queremos classificar (semelhante a um exemplo de um SVM com kernel mais robusta).

As **funções de ativação mais comuns** são as seguintes²:

- **Sigmoide:**

$$\text{sigm}(z) = \frac{1}{1 + e^{-z}} \iff \text{sigm}'(z) = \text{sigm}(z) \times \text{sigm}(1 - z)$$

- **Tangente hiperbólica:**

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \iff \tanh'(z) = 1 - \tanh(z)^2$$

- **Rectified Linear Unit (ReLU):**

$$\text{ReLU}(x) = \begin{cases} x, & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases} \iff \text{ReLU}'(x) = \begin{cases} 1, & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases}$$

ou seja, os valores negativos tomam o valor 0 e os valores positivos tomam o seu valor x ou 1.

A **arquitetura de uma rede neuronal** tipicamente tem três camadas:

- **uma camada de entrada** onde os valores dos atributos de cada amostra entram na rede (representação visual para fazer ligação dos dados de entrada às outras camadas da rede)
- **camadas "escondidas"** compostas por um nº arbitrário de neurónios. Os neurónios da camada de entrada estão ligados a todos os pontos desta camada e, por isso, chama-se a esta camada de **Densa** ou **completamente ligada**
- **uma camada de saída** que tem o número de neurónios fixado consoante o problema:
 - **Estimação de um valor/Regressão** - um único neurónio de saída, que **não tem função de ativação** ou que pode ter **função de ativação ReLu** se a predição só puder assumir valores positivos (exemplo: preço de uma casa, idade de uma pessoa, etc.)
 - **Classificação binária** - um único neurónio de saída, com uma função de ativação do tipo **sigmoide** ou **tanh**
 - **Classificação multi-classe** - tantos neurónios de saída, quanto o número de classes presentes no problema. Normalmente tem uma função de ativação "softmax" para ter valores normalizados e estes serem interpretados como probabilidades

²para reduzir a complexidade no treino das redes, as funções de ativação mais usadas têm derivadas que são mais simples matematicamente

Ao treinar uma rede neuronal dividimos os dados em três conjuntos: treino, para otimizar os pesos da rede; validação, para ir avaliando o desempenho do treino; teste, para medir o desempenho dados novos, desconhecidos para o modelo.

A otimização dos pesos é feita por um algoritmo de **back propagation**, que utiliza uma função de perda que quantifica o erro das predições e procura iterativamente o conjunto de pesos que minimiza esta função de perda.

As funções de perda mais comuns são as seguintes:

- **Mean square error** - usada em Regressão

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Binary cross-entropy** - usada em classificação binária

$$L_{BCE} = - \sum_{i=1}^N [y_i \times \log(\hat{y}_i) + (1 - y_i) \times \log(1 - \hat{y}_i)]$$

- **Categorical cross-entropy** - usada em classificação multi-classe

$$L_{CCE} = - \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \cdot \log(\hat{y}_{i,j})$$

Os termos variáveis da função de perda são os pesos, dado que x_i são os vetores de *features* das amostras e y_i são os valores verdadeiros. Os pesos ótimos minimizam a perda: $W^* = \operatorname{argmin} L(W)$. Algoritmo variante do Gradient Descent (GD).

As métricas de desempenho para problemas de classificação são as clássicas: $Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$; $Precision = \frac{TP}{TP + FP}$; $Recall = \frac{TP}{TP + FN}$.

Implementação no TensorFlow (pormenores) : A compilação da rede é feita com o `compile(...)` e aqui dizemos a função de perda, o otimizador (Adam, por exemplo, que é uma variação do GD) e as métricas a utilizar no treino. Para além disto, no `fit(...)` do modelo, o `batch_size` corresponde a quantas iterações por época serão realizadas, isto é, $\text{Iterações por época} = \frac{\text{nº amostras}_{\text{treino}}}{\text{batch size}} \longleftrightarrow \frac{2000}{32} = 62.5 \approx 63$ e as épocas é o número de vezes que itera sobre todas as amostras de treino (treinará para tudo se não forem utilizadas *callbacks* para parar o treino).

Para calcularmos o número de parâmetros treináveis numa rede neuronal clássica podemos fazer: $[(\text{nº neurónios camada de entrada} \times \text{nº neurónios camada densa}) + \text{nº neurónios camada densa}] + [(\text{nº neurónios camada densa} \times \text{nº neurónios camada de saída}) + \text{nº neurónios camada de saída}]$ (isto se a rede tiver uma camada de entrada, uma densa e uma de saída).

REDES NEURONAIS CONVOLUCIONAIS

Na abordagem clássica extraímos atributos da imagem e a rede produz predições com base nisso. Contudo, este processo é limitativo, pois é difícil de extrair *features* de alto nível, como a estrutura dos objetos.

A ideia principal das redes neurais convolucionais é tirar partido da estrutura espacial das imagens e ligar os pixels a neurónios, a fim de organizá-los por blocos (de dimensão pequena).

A arquitetura adequada para imagens é que cada um dos blocos é ligado a um neurónio, onde todos os neurónios que estão na mesma camada partilham os mesmos pesos nas ligações entre si. No fundo corresponde a uma convolução de uma imagem com um filtro.

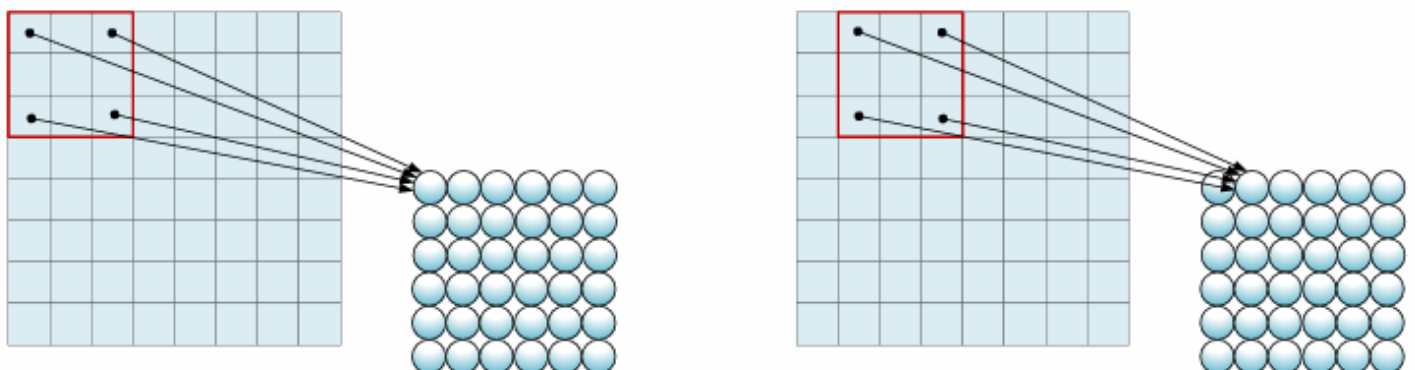


Figura 10: Arquitetura CNN

O resultado desta convolução é visto como um *feature map*. Para se obterem *feature maps* diferentes utilizamos filtros diferentes também; são tantos filtros quanto o número de *feature maps* que se pretende gerar através da camada convolucional.

A estrutura de uma CNN tem:

- **Camadas convolucionais** - aplicam os filtros para se obterem os *feature maps*. Os coeficientes dos filtros são pesos aprendidos na rede. Têm unidades de ativação ReLu para colocar 0 nos valores negativos, que representa vantagens relativamente à sigmoide porque é computacionalmente mais eficiente e evita um problema de "vanishing gradient"(gradientes com valores próximos de 0 que impede/atrasa convergência.
- **Camadas de sub-amostragem/pooling** - reduzem a dimensão dos *feature maps* e diminuir o nº de operações em camadas subsequentes. Capta as partes mais importantes dos mapas e introduz alguma robustez em relação a pequenas translações nas imagens
- **Camadas densas** - idênticas à camada "escondida" numa rede neuronal clássica, que na prática funcionam como um classificador (camada completamente ligada e camada de saída)

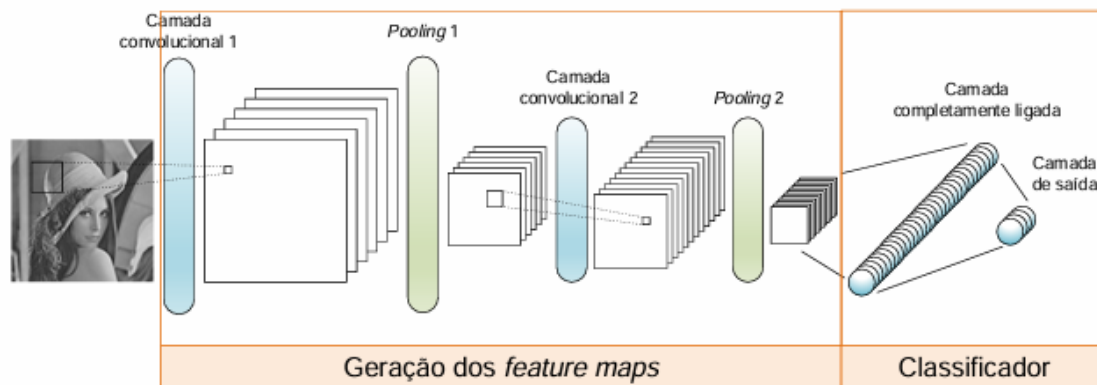


Figura 11: Estrutura de CNN para classificação

OVERFITTING E FORMAS DE REDUZIR

Sobre-ajuste/Overfitting é uma característica que ocorre quando a rede tem um bom desempenho nos dados de treino, mas não consegue generalizar para os dados de validação e teste. Este fenómeno pode ser detetado observando a evolução da função de perda ao longo do treino e evitado ao:

- **Simplificar o modelo**
- Utilizar técnicas de **regularização na função de perda**
- Usar técnicas de **data augmentation** (DA) (novas versões das imagens de treino), como zoom in/out, "flips", rotações, introdução de ruído, controlo de contraste e luminosidade, etc.
- Utilizar uma camada de **dropout** que desliga neurónios da rede aleatoriamente

MODELOS PRÉ-TREINADOS

A 1ª CNN foi a **Neocognitron** e foi proposta em 1980, contudo, a Alex Net (60M params) foi a que trouxe mais notoriedade às CNNs ao vencer a competição de classificação do ImageNet, em 2012. Utilizou ativação ReLu e DA e dropout para evitar overfitting.

Um **modelo pré-treinado** é uma rede CNN mais complexa que foi treinada para resolver problemas de classificação similar ao que queremos. Podemos aplicá-lo diretamente (`decode_predictions(...)`), se as classes do problema estiverem incluídas no treino original, ou utilizando transferência de conhecimento(`Model.trainable=False`), se tivermos classes diferentes das usadas no treino das redes.

Existem várias famílias de redes pré-treinadas :

- **VGG-??**: Rede que venceu a competição do ImageNet em 2014, onde os "??"correspondem ao número de camadas e em que as principais diferenças face à AlexNet é que os filtros são 3x3 e utilizar várias camadas convolucionais consecutivas sem *pooling* entre elas

- **ResNet**: introduziram o conceito de *skip-connection* em que o input de um módulo de camadas conv. tem um caminho em paralelo que permite saltar o módulo. A vantagem é a de evitar o "vanishing gradient" em redes com muita profundidade
- **MobileNet**: introduziram o conceito de convolução separável, que reduz o nº de operações aritméticas relacionadas com a convolução à custa de uma degradação no desempenho

Na transferência de conhecimento treina-se apenas a parte do novo classificador, ou seja, para um problema diferente com classes diferentes.

DETEÇÃO DE OBJETOS

O principal objetivo é obter a localização, numa bounding box, e identificar objetos nas imagens/frames de vídeos.



Figura 12: Segmentação e deteção de objetos em imagens

Para deteção de objetos existem soluções:

- **com duas etapas**
 - **R-CNN (2014)**: é usado um algoritmo para obter regiões candidatas à classificação e depois, num 2º passo, usa uma CNN para classificar regiões obtidas com base no grau de confiança
 - **Fast R-CNN e Faster R-CNN (2015)**: Fast - as convoluções são tratadas em conjunto e não região a região, como na R-CNN original; Faster - inclui uma rede neuronal dedicada (RPN)³ para obter as candidatas e usa essa rede como input dos *feature maps*
- **integradas**
 - **SSD (2016)**: as regiões são extraídas através dos *feature maps* e usa um conjunto de dimensões pré-definidas para as bounding boxes
 - **YOLO - You Only Look Once**: a rede neuronal é diretamente treinada para gerar bounding-boxes de possíveis objetos e as suas classes, onde as com maiores graus de confiança são consideradas objetos detetados. Utiliza uma **backbone**, para extrair os *feature maps*; uma **neck**, que faz a fusão nos mapas gerados pelo backbone a diferentes escalas; e uma **head**, com o intuito de produzir as deteções e que é composta por regressores de bounding boxes e preditores de classes

Para avaliar o desempenho podemos utilizar o Intersection over Union (IoU) para ver se a bounding box é adequada, através do seguinte cálculo:

$$\text{IoU} = \frac{A \cap B}{A \cup B} = \frac{A \cap B}{A + B - A \cap B}$$

Para além disto podemos calcular a Average Precision (AP), que é uma aproximação da área por baixo da curva Precision-Recall, e, por conseguinte, a mean Average Precision (mAP) - média da AP para as várias classes.

³Region Proposal Network